

VBLS: VIT-BASED BROAD LEARNING SYSTEM FOR REMOTE SENSING SCENE CLASSIFICATION

Zishu Liu[✉]

University of Exeter
Exeter, United Kingdom
z1538@exeter.ac.uk

Chunbo Luo*

University of Exeter
Exeter, United Kingdom
C.Luo@exeter.ac.uk

Zeyu Fu[✉]

University of Exeter
Exeter, United Kingdom
Z.Fu@exeter.ac.uk

David Moffat[✉]

Plymouth Marine Lab
Plymouth, United Kingdom
dmof@pml.ac.uk

Abstract—The Broad learning system (BLS) has been used for image classification tasks due to its efficient training and evaluation progress. However, BLS encounters challenges in remote sensing scene classification tasks due to its unclear parameter configuration and the large image size. To address these limitations, we propose the VBLS model which takes tokens from a vision transformer instead of raw images as input, thereby reducing the computational cost of vanilla BLS while capturing both local features and global relationships. Moreover, key BLS parameters can be directly defined by the number of tokens, simplifying the model configuration. The proposed model with total time consumption of 106.07 seconds and 370.05 seconds without GPU acceleration on two public datasets AID and EuroSAT, respectively. These results represent an order of magnitude faster than deep learning models using the same computing configurations, while maintaining comparable classification accuracies.

Index Terms—Broad learning system, remote sensing, scene classification, vision transformer

I. INTRODUCTION

Remote sensing (RS) scene classification is an essential step in various Earth observation tasks such as precision agriculture [1] and urban planning [2]. As the data size and complexity of the RS images grow, traditional handcrafted feature descriptors [3] are incapable of capturing complex spatial relationships in RS images and often lack generalization ability between diverse RS images, which limits their application to complex RS data.

Researchers introduced many convolutional neuron networks (CNNs) into RS image scene classification tasks and achieved promising results [4]. As datasets for RS scene classification usually include fewer samples than those for traditional image classification, training CNN models from scratch can be difficult. Thus, the researchers proposed strategies for fine-tuning pre-trained models on RS datasets [5]. In addition to fine-tuning pre-trained models, the performance of CNNs can be improved by capturing and fusing features from different levels of RS images [6]. Although CNNs achieved promising classification accuracy on some RS datasets, they usually neglected long-range contextual relationships in RS

images, which are particularly important for scene classification tasks. On the other hand, the vision transformer (ViT) [7] captured long-range contextual relationships in RS images by introducing self-attention layers. Thus, scholars combined ViT with CNN in various ways to capture local and long-range information in RS images [8].

Although both CNN-based models and ViT-based models achieved promising results in RS scene classification tasks, they require extensive training due to their complex architectures [9] and require GPUs acceleration. However, in many real-world deployment scenarios for RS data analysis, such as edge devices, embedded systems, and large-scale online inference services, GPUs are often unavailable or impractical due to constraints on power consumption, physical space, and system compatibility. Consequently, the machine learning models in these systems must be executed efficiently on CPUs. In addition, the vanilla ViT calculates tokens for all patches in input images but only uses the class token for classification [7]. Thus, further explorations are required on how to fully utilize the output of the ViT's encoder.

The broad learning system (BLS) [10] offers fast training and inference due to its simple structure and training algorithm. BLS has been used as a classifier in RS applications when combined with deep learning models [11]. However, BLS's performance is sensitive to its parameter settings. Researchers often carried out many trials to find optimal settings [11] due to the lack of systematic methodologies [12]. BLS also requires proper pre-processing to avoid potential issues of excessive memory usage when processing RS images.

II. PROPOSED METHOD

To address the challenges faced by BLS in RS images while maintaining its efficiency during training and evaluation, we introduce a ViT encoder to reduce the memory usage in BLS during the feature mapping stage by decreasing the size of the weight matrices W_i . The incorporation of the ViT encoder also helps define parameters in BLS. The proposed model can fully exploit all tokens from the ViT encoder while demonstrating an order of magnitude reduction in training time compared to deep learning models using the same computing configuration.

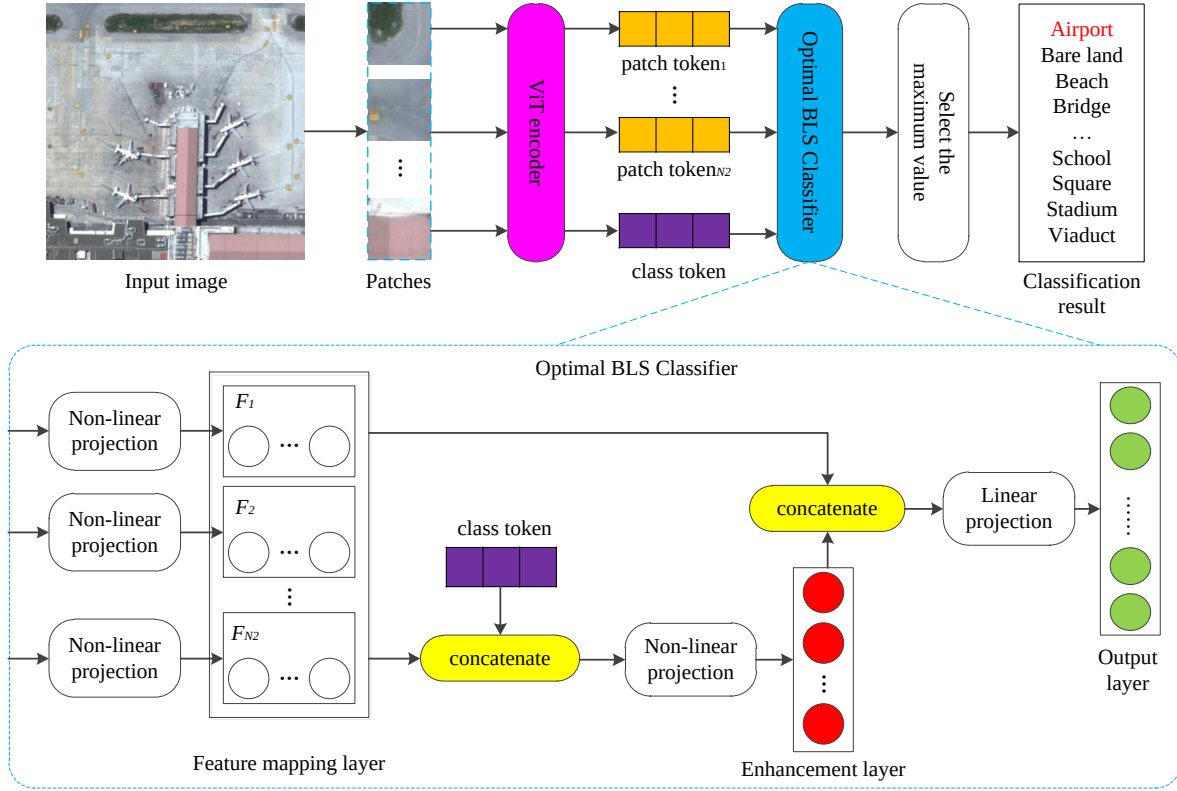


Fig. 1. Structure for the proposed model.

We also present an optimization strategy for the BLS structure to optimally set the number of windows in the feature mapping layer.

A. Model Overview

The proposed model is designed to reduce the training time of ViT and improve the performance of the BLS model on RS images. As shown in Fig. 1, our model introduces a BLS classifier that can efficiently process all tokens from the ViT encoder. The input images are first resized to 224×224 then processed by a pre-trained ViT encoder and get a 3D matrix with shape $[B, N_2+1, K]$ where B is the batch size, N_2+1 is the number of tokens (N_2 tokens from patches and one class token), and K is the length of each token. In our model, the feature mapping layer extracts high-level features from patch tokens using (1) where X_i is the patch token, W_i is a randomly generated weight matrix and F_i is the i^{th} feature window in the feature mapping layer. Since the class token encodes the global relationship, it is not suitable for the role of the feature mapping layer, which focuses on local feature processing. So, the number of windows in the feature mapping layer is set to the number of patch tokens N_2 .

$$F_i = \xi(X_i W_i + \beta_i), i = 1, 2, 3, \dots, N_2 \quad (1)$$

The local information from the feature mapping layer does not contain the global relationship between patches. Hence, the class token concatenates the output of the feature mapping

layer $F = [F_1, F_2, \dots, F_{N_2}]$, forming a new matrix $F' = [F | class_token]$ that contains both local information and their global relationships. The enhancement layer E is calculated by (2), where $\phi(\cdot)$ is a non-linear activation function, and W' and β' are a randomly generated weight matrix and bias vector, respectively. Then, we concatenate outputs from the feature mapping layer and the enhancement layer, and calculate final classification results like vanilla BLS.

$$E = \phi(F' W' + \beta') \quad (2)$$

B. Feedforward and Training

Denote the input matrix of BLS as $X \in R^{N \times M}$ that contains N samples with M features in each sample. The feature mapping layer $F = [F_1, F_2, \dots, F_{N_2}]$ is calculated by (1) from input matrix, where F_i is the i^{th} window in the feature mapping layer, W_i and β_i are a randomly generated weight matrix and bias vector for each window, respectively. A sparse autoencoder is introduced to update W_i to improve the ability to extract features. The enhancement layer $E = [E_1, E_2, \dots, E_{N_3}]$ are calculated by (2).

Denoting $H = [F | E]$, the output Y for BLS is calculated by (3) where the weight matrix W is the target for training BLS.

$$Y = HW \quad (3)$$

Performing directly calculating the pseudo-inverse matrix H^+ can be difficult because its size is usually large. Thus,

BLS introduces the ridge regression algorithm shown in (4) to approximate H^+ where λ is a number for regulation and I is an identity matrix. The final step for BLS training is to update the weight matrix W using (5).

$$H^+ = (\lambda I + HH^T)^{-1} H^T \quad (4)$$

$$W = H^+ Y = (\lambda I + HH^T)^{-1} H^T Y \quad (5)$$

C. Optimization Strategy

The efficiency of BLS is affected by three key parameters: the number of windows in the feature mapping layer (N_2), the number of nodes in each window (N_1), and the number of nodes in the enhancement layer (N_3). These parameters affect the size of weight matrices and bias vectors in BLS. However, different combinations of N_1 and N_2 will lead to different time consumption in BLS, even the number of nodes in the feature mapping layer is the same. This variability further increases the difficulty of finding an optimal BLS configuration for RS scene classification tasks. We propose iterative optimization of the three parameters as follows. Firstly, we need to identify the parameter that affects the BLS's efficiency most in order to improve the overall performance of the model. To find this parameter, the algorithm evaluates the time consumption of the BLS classifier under different parameter settings. When increasing one of the parameters, the other two parameters remain unchanged. Based on the BLS configurations in [11] and [13], we define the parameter search space shown in Table I where each row varies one parameter while fixing the others, with the specified step size.

TABLE I
PARAMETER SETTING FOR BLS

N_1	N_2	N_3	Increment per step
10 $\rightarrow$ 200	20	2000	10
200	1 $\rightarrow$ 20	2000	1
200	20	100 $\rightarrow$ 2000	100

We use the RSSCN7 [14] dataset (2800 RGB images with a resolution of 400×400 pixels) as an example to demonstrate the optimization strategy, the results are shown in Fig. 2. The time consumption of the classifier increases rapidly when there are more nodes in the feature mapping layer, while increasing the enhancement nodes (N_3) does not bring significant changes in time consumption (fluctuates around 70 seconds). When comparing the trend of the curves shown in Fig.2 and the time consumed by the BLS, adding more windows (N_2) to the feature mapping layer leads to higher time consumption and a steeper growth in processing time. Thus, setting N_2 to a specific number will weaken the impact of structural changes in BLS's performance.

Instead of determining the most suitable N_2 for different datasets through extensive experiments, this paper proposes a more general method that can be applied to different datasets. We notice that the outputs of the ViT encoder have the shape of $[tokens, length]$. However, only the class token is used for

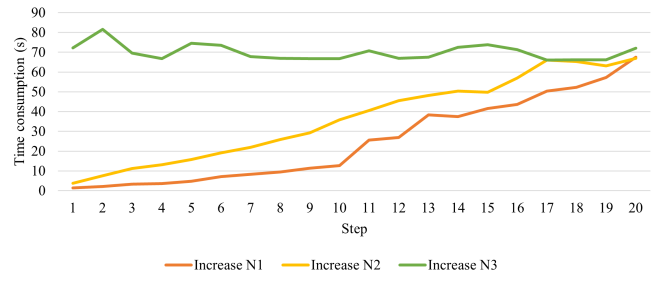


Fig. 2. Time consumption under different BLS structures.

classification, while other tokens are abandoned in the classification procedure. Since the local information in patch tokens cannot be ignored in RS image scene classification tasks, we utilize all tokens from ViT encoder in the proposed model, which ensures that BLS can capture both local information and global relationship within the input images.

Following this idea, the feature mapping layer in the proposed model will process tokens from patches. The enhancement layer, which integrates information from all feature windows, will process the global relationship. Treating all patch tokens as the input of BLS as vanilla BLS does may lead to an out-of-memory problem due to the large W_i ($tokens \times length \times N_1$) in (1). To address the above problems, one feature window in our model processes a single patch token from the ViT encoder, which means that each W_i only has $(length \times N_1)$ elements. Meanwhile, the parameter N_2 is defined as the number of patches in the ViT encoder.

III. EXPERIMENTS

A. Datasets and experiment settings

We evaluated our model on two widely used RS image datasets: AID [15] (10000 RGB images with a resolution of 600×600 pixels, 30 categories) and EuroSAT [16] (27000 RGB images with a resolution of 64×64 pixels, 10 categories). We selected 90% of the samples from each category as training sets, and the rest 10% of the samples form the evaluation set for each dataset.

All experiments were run on a computer with an Intel(R) Xeon(R) w5-2465X CPU and 134 GB RAM. The coding environment includes Ubuntu 24.04, Python 3.11, and Pytorch 2.4.0. We used ViT encoders pre-trained on the ImageNet-1K dataset [17] with two patch sizes 16×16 and 32×32 , and the length of each token is 768. In this section, we denote VBLS models with these encoders as VBLS (16×16) and VBLS (32×32), respectively. To highlight the computational performance of all models, we use the results obtained from CPU execution as a benchmark for empirical evaluation.

B. Feature Extractor Selection

Several ViT variations employ different patch sizes and token lengths [7], which affect the our model's performance by influencing the parameter N_2 and the size of W_i . Thus, we need to find the optimal feature extractors for the proposed

TABLE II
FEATURE EXTRACTOR IMPACT

Dataset	Patch size	Patches	Token length	OA (%)	Time (s)
AID	16 × 16	196	768	81.52 ± 0.31	481.32
	16 × 16	196	1024	86.36 ± 0.86	931.78
	32 × 32	49	768	84.60 ± 0.57	106.12
	32 × 32	49	1024	82.72 ± 0.31	210.10
	14 × 14	256	1280	88.24 ± 0.64	2298.26
EuroSAT	16 × 16	196	768	94.52 ± 0.18	1552.74
	16 × 16	196	1024	97.06 ± 0.21	2713.54
	32 × 32	49	768	95.53 ± 0.14	370.18
	32 × 32	49	1024	93.80 ± 0.09	623.07
	14 × 14	256	1280	94.76 ± 0.04	6716.17

TABLE III
BLS PARAMETER CONFIGURATION

Model	N_1	N_2	N_3
VBLS (16 × 16)	10	196	1000
VBLS (32 × 32)	10	49	1000
Vanilla BLS	150	20	1000

model by comparing the overall accuracy (OA) and the time consumption on the target datasets. The results shown in Table II are the average of 5 experiments. We denote those ViT variants using their patch size and token length (B for models with 768 token length, L for models with 1024 token length, and H for models with 1280 token length). For example, ViT-B/16 means the ViT feature extractor module with 16 × 16 patch size and 768 token length.

Compared to models that use ViT-B feature extractors, those employing ViT-L extractors with the same patch size require roughly twice the computation time across all datasets. Models that use ViT-L/16 achieve a higher OA in the two datasets, but their time consumption is 9 times higher than those that use ViT-B/32 feature extractor due to the larger parameter size of the ViT-L/16 architecture [7]. In contrast, ViT-B/32 offers a better trade-off between OA and computational cost. Models with the ViT-H/14 feature extractor spend the longest time and do not significantly improve OA. Since our goal is to reduce the model’s time consumption, the ViT-H/14 feature extractor is not suitable for our model. Therefore, we adopt ViT-B/16 and ViT-B/32 as feature extractors in the proposed VBLS.

C. Classification Results

In this section, we compare the performance of VBLS with other deep learning models in OA and time consumption. All deep learning models are pre-trained on ImageNet-1K and fine-tuned on the target datasets. The time consumption for deep learning models is the first time that they achieved the highest OA. The configurations of different BLS classifiers are listed in Table III. Experimental results are shown in Table IV, where OA and time consumption listed in the table are the average values over 5 experiments.

Compared to deep learning models, VBLS significantly reduces the time consumption in the AID and EuroSAT datasets. VBLS is nearly 8 times faster than the fastest deep learning model ResNet18 [18] on AID, and ranks as the second

TABLE IV
PERFORMANCE ON DIFFERENT DATASETS

Model	AID		EuroSAT	
	Time (s)	OA (%)	Time (s)	OA (%)
ResNet18 [18]	816.50	82.3 ± 0.28	176.77	86.64 ± 0.12
ResNet50 [18]	2541.64	83.42 ± 0.29	746.07	90.30 ± 0.40
WideResNet50 [19]	3845.04	81.48 ± 0.16	1707.90	83.79 ± 0.32
ViT-B/16 [7]	5531.32	82.96 ± 0.002	13464.34	94.77 ± 0.001
ViT-B/32 [7]	1043.11	84.90 ± 0.002	1510.12	96.22 ± 0.0007
MaxViT [20]	4309.35	84.12 ± 0.52	2977.06	95.41 ± 0.15
DeiT [21]	5839.24	84.10 ± 0.06	23053.33	95.07 ± 0.04
MobileNetV2 [22]	1958.95	83.18 ± 0.86	4242.07	94.96 ± 0.19
BLS [10]	529.18	19.50 ± 0.0022	377.05	36.24 ± 0.0007
VBLS (16 × 16)	481.32	81.52 ± 0.31	1552.74	94.52 ± 0.18
VBLS (32 × 32)	106.12	84.60 ± 0.57	370.18	95.53 ± 0.14

fastest model on EuroSAT. Although ResNet18 halves the time consumption to 176.77 seconds for training and evaluation, its OA is approximately 10% lower than that of our model. Compared to the light-weighted MobileNetV2 model [22], our model is 10 times faster while slightly improving OA in two datasets. Our model achieves the second-highest OA in two datasets, slightly lower than that of ViT-B/32, while offering a favorable trade-off between accuracy and efficiency.

In terms of different feature extractors, VBLS (32 × 32) is five times faster than VBLS (16 × 16) and achieves a higher OA in all three datasets. As VBLS (16 × 16) processes nearly five times more patch tokens than VBLS (32 × 32), we think this is the reason for the difference in efficiency. Table V shows the time consumption of our model in detail. Indeed, VBLS (16 × 16) spends three to five times longer than VBLS (32 × 32) in all three stages. VBLS (16 × 16) achieves a lower OA than VBLS (32 × 32), which implies that excessively increasing the number of nodes in the feature mapping layer can have a negative influence on the model’s performance.

TABLE V
DETAILED TIME DECOMPOSITION

Dataset	Model	Extraction (s)	Training (s)	Evaluation (s)
AID	VBLS (32 × 32)	104.74	1.33	0.05
	VBLS (16 × 16)	477.59	3.58	0.15
EuroSAT	VBLS (32 × 32)	367.39	2.66	0.13
	VBLS (16 × 16)	1544.19	8.10	0.45

IV. CONCLUSION

In this paper, we proposed a novel model for efficient RS scene classification. The proposed model defined the number of feature window (N_2) in BLS with the number of patches of the ViT encoder, which provided an optimal method for the configuration of the BLS parameters. The BLS classifier can efficiently use all the tokens from the ViT encoder instead of only using the class token, which avoided potential information loss. The proposed model worked at least five times faster than many deep learning models on two public RS datasets AID and EuroSAT, and achieved a classification accuracy of 84.6% and 95.53%, respectively.

REFERENCES

- [1] R. P. Sishodia, R. L. Ray, and S. K. Singh, "Applications of remote sensing in precision agriculture: A review," *Remote Sens.*, vol. 12, no. 19, 2020. [Online]. Available: <https://www.mdpi.com/2072-4292/12/19/3136>
- [2] J. Li, U. A. Bhatti, S. A. Nawaz, M. Huang, R. M. Ahmad, and Y. Y. Ghadi, "Remote-sensing image classification: A comprehensive review and applications," *Deep Learning for Multimedia Processing Applications*, pp. 18–47, 2024.
- [3] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *Int. J. Comput. Vis.*, vol. 60, pp. 91–110, 2004.
- [4] M. Mehmood, A. Shahzad, B. Zafar, A. Shabbir, and N. Ali, "Remote sensing image classification: A comprehensive review and applications," *Math. Probl. Eng.*, vol. 2022, no. 1, p. 5880959, 2022.
- [5] C. Broni-Bediako, Y. Murata, L. H. B. Mormille, and M. Atsumi, "Searching for cnn architectures for remote sensing scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, pp. 1–13, 2022.
- [6] J. Wang, W. Li, M. Zhang, R. Tao, and J. Chanussot, "Remote-sensing scene classification via multistage self-guided separation network," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–12, 2023.
- [7] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," *arXiv preprint arXiv:2010.11929*, 2020.
- [8] D. Wang, Q. Zhang, Y. Xu, J. Zhang, B. Du, D. Tao, and L. Zhang, "Advancing plain vision transformer toward remote sensing foundation model," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–15, 2023.
- [9] B. Liu, C. Zhan, C. Guo, X. Liu, and S. Ruan, "Efficient remote sensing image classification using the novel stconvnext convolutional network," *Sci. Rep.*, vol. 15, no. 1, p. 8406, 2025.
- [10] C. P. Chen and Z. Liu, "Broad learning system: An effective and efficient incremental learning system without the need for deep architecture," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 29, no. 1, pp. 10–24, 2017.
- [11] S. Wang, K.-V. Yuen, X. Yang, and Y. Zhang, "Tropical cyclogenesis detection from remotely sensed sea surface winds using graphical and statistical features-based broad learning system," *IEEE Trans. Geosci. Remote Sens.*, vol. 61, pp. 1–15, 2023.
- [12] Y. Chu, Y. Guo, W. Ding, H. Cao, and P. Ping, "Broad learning systems: An overview of recent advances, applications, challenges and future directions," *Neurocomputing*, p. 130337, 2025.
- [13] G. Zhao, X. Wang, and Y. Cheng, "Hyperspectral image classification based on local binary pattern and broad learning system," *Int. J. Remote Sens.*, vol. 41, no. 24, pp. 9393–9417, 2020.
- [14] Q. Zou, L. Ni, T. Zhang, and Q. Wang, "Deep learning based feature selection for remote sensing scene classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 12, no. 11, pp. 2321–2325, 2015.
- [15] G.-S. Xia, J. Hu, F. Hu, B. Shi, X. Bai, Y. Zhong, L. Zhang, and X. Lu, "Aid: A benchmark data set for performance evaluation of aerial scene classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 7, pp. 3965–3981, 2017.
- [16] P. Helber, B. Bischke, A. Dengel, and D. Borth, "Eurosat: A novel dataset and deep learning benchmark for land use and land cover classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 12, no. 7, pp. 2217–2226, 2019.
- [17] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein *et al.*, "Imagenet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, pp. 211–252, 2015.
- [18] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [19] S. Zagoruyko and N. Komodakis, "Wide residual networks," *arXiv preprint arXiv:1605.07146*, 2016.
- [20] Z. Tu, H. Talebi, H. Zhang, F. Yang, P. Milanfar, A. Bovik, and Y. Li, "Maxvit: Multi-axis vision transformer," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*. Springer, 2022, pp. 459–479.
- [21] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. Proc. Mach. Learn. Res., M. Meila and T. Zhang, Eds., vol. 139. PMLR, 18–24 Jul 2021, pp. 10 347–10 357.
- [22] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510–4520.